AI Assisted Coding
Assignment - 12.4
2303A510B7
Batch - 14

Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Task 1: Bubble Sort for Ranking Exam Scores

Prompt :

Write a Python program to implement Bubble Sort for sorting a list of student exam scores.
Requirements:
1. Use Bubble Sort algorithm only.
2. Add clear inline comments explaining:
   - Each comparison between elements
   - When and why swapping happens
   - Each outer loop pass
3. Add an early termination optimization:
   - If no swaps occur in a pass, stop the algorithm early.
4. After the function, include:
   - A short time complexity analysis (Best, Average, Worst case)
   - Explanation of when Bubble Sort is suitable.
5. Include sample input and output demonstration.
6. Keep the code beginner-friendly and readable.
7.User defined input for scores.
Do NOT use built-in sort() or sorted() functions.

Code :

AI-12.4.py > ...

```python
'''Write a Python program to implement Bubble Sort for sorting a list of student exam scores.
Requirements:
1. Use Bubble Sort algorithm only.
2. Add clear inline comments explaining:
   - Each comparison between elements
   - When and why swapping happens
   - Each outer loop pass
3. Add an early termination optimization:
   - If no swaps occur in a pass, stop the algorithm early.
4. After the function, include:
   - A short time complexity analysis (Best, Average, Worst case)
   - Explanation of when Bubble Sort is suitable.
5. Include sample input and output demonstration.
6. Keep the code beginner-friendly and readable.
7.user defined input for scores.
Do NOT use built-in sort() or sorted() functions.'''

def bubble_sort(scores):
    n = len(scores)
    # Outer loop to traverse through all elements
    for i in range(n):
        # Flag to check if any swapping occurs
        swapped = False

        # Inner loop to compare adjacent elements
        for j in range(0, n-i-1):
            # Compare the current element with the next element
            if scores[j] > scores[j+1]:
                # If the current element is greater, swap them
                scores[j], scores[j+1] = scores[j+1], scores[j]
                swapped = True  # Set flag to True if a swap occurred

        # If no swapping occurred, the list is already sorted
        if not swapped:
            break  # Exit the loop early
# Time Complexity Analysis:
# Best Case: O(n) - when the list is already sorted (only one pass needed)
# Average Case: O(n^2) - when the list is in random order
# Worst Case: O(n^2) - when the list is sorted in reverse order
# Bubble Sort is suitable for small datasets or when the list is nearly sorted.
# Sample Input and Output Demonstration
if __name__ == "__main__":
    # User-defined input for scores
    scores_input = input("Enter student exam scores separated by commas: ")
    scores = [int(score.strip()) for score in scores_input.split(",")]

    print("Original Scores:", scores)
    bubble_sort(scores)
    print("Sorted Scores:", scores)
```

OutPut :

```
Enter student exam scores separated by commas: 90,88,75,60,80,99
Original Scores: [90, 88, 75, 60, 80, 99]
Sorted Scores: [60, 75, 80, 88, 90, 99]
(base) harini@HARINIs-MacBook-Air AI assisted coding %
```

Prompt :

Explain why early termination improves performance in best-case scenarios.

Explanation :

Early termination improves performance in best-case scenarios by allowing an algorithm to stop execution as soon as the desired condition is met, rather than continuing unnecessary iterations or computations. This reduces the time complexity in cases where the input data is already sorted, partially processed, or meets the criteria early in the process.

For example, in a **Bubble Sort** algorithm, early termination can be implemented by checking if no swaps occurred during a pass. If no swaps are made, the list is already sorted, and the algorithm can terminate early without completing all iterations. This optimization avoids redundant work and improves performance in best-case scenarios, reducing the time complexity from $O(n^2)$ to $O(n)$ for sorted inputs.

Task 2: Improving Sorting for Nearly Sorted Attendance Records

Prompt :

You are helping me with a Data Structures lab assignment.
Scenario:
I am maintaining an attendance system where student roll numbers are nearly sorted, with only a few late updates.
Task Requirements:
1. First, implement Bubble Sort in Python to sort a list of roll numbers.
   - Add clear inline comments explaining comparisons and swaps.
   - Include early termination optimization.
2. Then review the scenario and suggest whether Bubble Sort is the best choice.
   - If not, recommend a more suitable sorting algorithm for nearly sorted data.
   - Explain the reasoning clearly.
3. Implement Insertion Sort in Python.
   - Add detailed inline comments explaining shifting and insertion logic.
   - Keep the code beginner-friendly.
4. Compare Bubble Sort and Insertion Sort on nearly sorted input:
   - Explain their behavior
   - Mention best, average, and worst-case time complexity
   - Highlight why one performs better for partially sorted datasets.
5. Provide a sample nearly sorted input and show expected output.
Do NOT use built-in sort() or sorted() functions.

Code :

AI-12.4.py > ...

```python
# 1. Implementing Bubble Sort
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the list
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n - i - 1):
            # Compare adjacent elements
            if arr[j] > arr[j + 1]:
                # If they are in the wrong order, swap them
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True  # Set the flag to True if a swap occurred
        # If no swapping occurred, the list is already sorted
        if not swapped:
            break
# 2. Reviewing the scenario and suggesting a better sorting algorithm
# Bubble Sort is not the best choice for nearly sorted data because it has a time complexity of O(n^2) in the worst case.
# A more suitable sorting algorithm for nearly sorted data is Insertion Sort, which has a time complexity of O(n) in the
# best case when the data is already sorted or nearly sorted.
# 3. Implementing Insertion Sort
def insertion_sort(arr):
    # Traverse through 1 to len(arr)
    for i in range(1, len(arr)):
        key = arr[i]  # The current element to be compared
        j = i - 1  # The index of the last sorted element
        # Move elements of arr[0..i-1], that are greater than key,
        # to one position ahead of their current position
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]  # Shift the element one position to the right
            j -= 1  # Move to the next element on the left
        arr[j + 1] = key  # Insert the key at its correct position in the sorted subarray
# 4. Comparing Bubble Sort and Insertion Sort on nearly sorted input
# Bubble Sort has a best-case time complexity of O(n) when the list is already sorted, but it can degrade to O(n^2) in the worst case.
# Insertion Sort has a best-case time complexity of O(n) when the list is already sorted or nearly sorted, and it can degrade to O(n^2) in the worst case.
# Insertion Sort performs better for partially sorted datasets because it minimizes the number of comparisons and shifts needed to place elements in their
# correct positions, while Bubble Sort will still require multiple passes through the list.
# 5. Sample nearly sorted input and expected output
nearly_sorted_input = [1, 2, 3, 5, 4, 6, 7]
print("Original nearly sorted input:", nearly_sorted_input)
# Sorting using Bubble Sort
bubble_sort(nearly_sorted_input)
print("Sorted using Bubble Sort:", nearly_sorted_input)
# Resetting the input for Insertion Sort
nearly_sorted_input = [1, 2, 3, 5, 4, 6, 7]
# Sorting using Insertion Sort
insertion_sort(nearly_sorted_input)
print("Sorted using Insertion Sort:", nearly_sorted_input)
# 6. User-defined input for roll numbers and demonstration of sorting process
user_input = input("Enter roll numbers separated by commas: ")
roll_numbers = list(map(int, user_input.split(',')))
print("Original user input:", roll_numbers)
# Sorting using Bubble Sort
bubble_sort(roll_numbers)
print("Sorted using Bubble Sort:", roll_numbers)
# Resetting the input for Insertion Sort
roll_numbers = list(map(int, user_input.split(',')))
# Sorting using Insertion Sort
insertion_sort(roll_numbers)
print("Sorted using Insertion Sort:", roll_numbers)
```

OutPut :

```
Original nearly sorted input: [1, 2, 3, 5, 4, 6, 7]
Sorted using Bubble Sort: [1, 2, 3, 4, 5, 6, 7]
Sorted using Insertion Sort: [1, 2, 3, 4, 5, 6, 7]
Enter roll numbers separated by commas: 19,90,56,35,76,89
Original user input: [19, 90, 56, 35, 76, 89]
Sorted using Bubble Sort: [19, 35, 56, 76, 89, 90]
Sorted using Insertion Sort: [19, 35, 56, 76, 89, 90]
(base) harini@HARINIs-MacBook-Air AI assisted coding %
```

Prompt :

Explain mathematically why Insertion Sort runs in nearly O(n) time for nearly sorted arrays.

Explanation :

Insertion Sort runs in nearly O(n) time for nearly sorted arrays because the number of comparisons and shifts required to insert each element into its correct position is minimal when the array is already close to being sorted. Here's the mathematical reasoning:

Key Observations:

Insertion Sort Mechanism:

For each element in the array, Insertion Sort compares it with the elements in the sorted portion of the array (to its left) and shifts larger elements to the right until the correct position for the current element is found.

Nearly Sorted Arrays:

In a nearly sorted array, most elements are already close to their correct positions. This means the number of comparisons and shifts required for each element is small.

Mathematical Analysis:

Let the array have n elements.

For each element at index i (starting from i = 1), the algorithm performs comparisons and shifts until the element is placed in its correct position.

If the array is nearly sorted, the number of comparisons and shifts for each element is proportional to the distance the element needs to move to reach its correct position.
Total Work Done:
Suppose the maximum distance any element needs to move is k (where k is small for nearly sorted arrays).
For each element, the number of comparisons and shifts is at most k.
The total work done is approximately: $$ \text{Total Comparisons/Shifts} = \sum_{i=1}^{n} O(k) = O(n \cdot k) $$
Since k is small (e.g., constant or logarithmic in n), the time complexity becomes: $$ O(n \cdot k) \approx O(n) $$
Best Case (Fully Sorted Array):
If the array is fully sorted, each element is already in its correct position, so no shifts are required.
The algorithm only performs one comparison per element, resulting in: $$ O(n) $$
Conclusion:
For nearly sorted arrays, the number of shifts and comparisons is minimal because elements are close to their correct positions. This reduces the overall time complexity of Insertion Sort to nearly O(n), making it highly efficient for such cases.

Task 3: Searching Student Records in a Database

You are helping me complete a Data Structures lab assignment.
Scenario:
I am developing a student information portal where users search for student records using roll numbers.
Task Requirements:
1. Implement Linear Search in Python:
   - It should work for unsorted student roll number data.
   - Add proper docstrings explaining:
      * Parameters
      * Return values
      * What the function does
   - Add inline comments explaining each step.
2. Implement Binary Search in Python:
   - It should work only on sorted roll number data.
   - Add proper docstrings explaining:

    * Parameters
    * Return values
    * Precondition (list must be sorted)
  - Add inline comments explaining how mid is calculated and how search space is reduced.
3. After the implementations, provide:
  - Clear explanation of when Binary Search is applicable.
  - Time complexity analysis:
    * Best, Average, Worst case for both algorithms.
  - Comparison of performance differences.
  - A short observation comparing results when using:
    * Unsorted list (Linear Search required)
    * Sorted list (Binary Search preferred)
4. Provide sample input and output demonstrations.
5. user defined inputs for testing both search algorithms.
Do NOT use built-in list index() method.
Keep the code beginner-friendly and readable.

Time complexity :

```
# Explanation of when Binary Search is applicable:
# Binary Search is applicable when the list of student roll numbers is sorted in ascending order.
# It is efficient for large datasets as it reduces the search space by half with each iteration,
# making it much faster than Linear Search for sorted data.
# Time complexity analysis:
# Linear Search:
# Best case: O(1) - when the target is the first element.
# Average case: O(n/2) - when the target is in the middle of the list.
# Worst case: O(n) - when the target is the last element or not present.
# Binary Search:
# Best case: O(1) - when the target is the middle element.
# Average case: O(log n) - when the target is found after several iterations.
# Worst case: O(log n) - when the target is not present in the list.
# Performance differences:
# Linear Search is straightforward and works on unsorted data but can be inefficient for large lists.
#  Binary Search is much faster for sorted data, but it requires the list to be sorted beforehand, which
#  can add overhead if the list is frequently updated.
# Observation comparing results:
# When using an unsorted list, Linear Search is necessary and will work correctly, but it may take longer
# to find the target. In contrast, when using a sorted list, Binary Search is preferred as it significantly
# reduces the time taken to find the target, especially as the size of the list increases.
```

Code :

2.4.py > binary_search

```python
def linear_search(arr, target):
    """
    Perform a linear search for the target in the given list.

    Parameters:
    arr (list): The list of student roll numbers to search through.
    target (int): The roll number to search for.

    Returns:
    int: The index of the target if found, otherwise -1.

    This function iterates through each element in the list and compares it with the target.
    If a match is found, it returns the index. If the end of the list is reached without finding the target, it returns -1.
    """
    for index in range(len(arr)):
        # Check if the current element matches the target
        if arr[index] == target:
            return index  # Target found, return its index
    return -1  # Target not found in the list
def binary_search(arr, target):
    """
    Perform a binary search for the target in the given sorted list.
    Parameters:
    arr (list): The sorted list of student roll numbers to search through.
    target (int): The roll number to search for.
    Returns:
    int: The index of the target if found, otherwise -1.
    Precondition: The list must be sorted in ascending order.
    This function uses a divide-and-conquer approach. It calculates the middle index and compares the middle element with the target.
    If the middle element matches the target, it returns the index. If the target is less than the middle element, it continues searching
    in the left half of the list; if greater, it continues in the right half. This process repeats until the target is found or the search space is exhausted.
    """
    left, right = 0, len(arr) - 1  # Initialize pointers for the search space
    while left <= right:
        mid = left + (right - left) // 2  # Calculate the middle index
        if arr[mid] == target:
            return mid  # Target found at index mid
        elif arr[mid] < target:
            left = mid + 1  # Target is in the right half, move left pointer
        else:
            right = mid - 1  # Target is in the left half, move right pointer
    return -1  # Target not found in the list
# Sample input and output demonstrations
if __name__ == "__main__":
    # Unsorted list for linear search
    unsorted_roll_numbers = [102, 101, 104, 103, 105]
    target_roll_number = 104
    result_linear = linear_search(unsorted_roll_numbers, target_roll_number)
    print(f"Linear Search: Target {target_roll_number} found at index {result_linear}" if result_linear != -1 else "Linear Search: Target not found")

    # Sorted list for binary search
    sorted_roll_numbers = sorted(unsorted_roll_numbers)  # Sort the list for binary search
    result_binary = binary_search(sorted_roll_numbers, target_roll_number)
    print(f"Binary Search: Target {target_roll_number} found at index {result_binary}" if result_binary != -1 else "Binary Search: Target not found")
    # User-defined inputs for testing both search algorithms
    user_input = int(input("Enter a roll number to search for: "))
    # Test linear search with user input
    result_linear_user = linear_search(unsorted_roll_numbers, user_input)
    print(f"Linear Search: Target {user_input} found at index {result_linear_user}" if result_linear_user != -1 else "Linear Search: Target not found")
    # Test binary search with user input
    result_binary_user = binary_search(sorted_roll_numbers, user_input)
    print(f"Binary Search: Target {user_input} found at index {result_binary_user}" if result_binary_user != -1 else "Binary Search: Target not found")
```

Out Put :

```
Linear Search: Target 104 found at index 2
Binary Search: Target 104 found at index 3
Enter a roll number to search for: 105
Linear Search: Target 105 found at index 4
Binary Search: Target 105 found at index 4
(base) harini@HARINIs-MacBook-Air AI assisted coding %
```

Prompt :

Why is Binary Search significantly faster than Linear Search for large datasets? Explain using time complexity growth comparison.

Explanation :

Binary Search is significantly faster than Linear Search for large datasets due to the difference in their time complexity growth rates:

Time Complexity of Linear Search:

Linear Search iterates through the dataset one element at a time until it finds the target or reaches the end of the dataset.

In the worst case, it has to check every element in the dataset, resulting in a time complexity of: [O(n)] where n is the number of elements in the dataset.

Time Complexity of Binary Search:

Binary Search works only on sorted datasets. It repeatedly divides the dataset in half, eliminating half of the remaining elements in each step.

In the worst case, the number of steps required to find the target is proportional to the number of times the dataset can be halved, which is: [O(\log_2(n))] where n is the number of elements in the dataset.

Growth Rate Comparison:

The key difference lies in how the two algorithms scale as the dataset size increases:

For Linear Search, the time required grows linearly with the size of the dataset. For example, doubling the dataset size doubles the number of comparisons.
For Binary Search, the time required grows logarithmically. Doubling the dataset size increases the number of steps by only 1 (since ($\log_2(2n) = \log_2(n) + 1$)).
Example:
Suppose the dataset has ($n = 1,000,000$) elements:
Linear Search may require up to 1,000,000 comparisons in the worst case.
Binary Search requires only: $$\log_2(1,000,000) \approx 20 \text{ comparisons.}$$

Conclusion:

The logarithmic growth of Binary Search makes it exponentially faster than Linear Search for large datasets, especially as the dataset size increases. This efficiency is why Binary Search is preferred for searching in sorted datasets. However, it requires the dataset to be sorted beforehand, which may add preprocessing time.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Prompt :

We are part of a data analytics team that needs to sort large datasets received in:
- Random order
- Already sorted order
- Reverse sorted order

Task Requirements:
1. I will provide partially written recursive functions for:
   - Quick Sort
   - Merge Sort
2. Complete the recursive logic for both algorithms.
3. Add meaningful docstrings for each function explaining:
   - Parameters
   - Return values

- How recursion is used
- Base condition

4. Add clear inline comments explaining:
   - Partition logic in Quick Sort
   - Divide and merge process in Merge Sort
   - How recursive calls reduce the problem size
5. After implementation, include:
   - Time complexity analysis (Best, Average, Worst case)
   - Space complexity comparison
   - Explanation of how recursion works in both algorithms
   - Performance discussion when tested on:
     * Random data
     * Already sorted data
     * Reverse sorted data
6. Provide sample test cases for:
   - Random list
   - Sorted list
   - Reverse sorted list

Do NOT use built-in sort() or sorted().
Keep code readable and beginner-friendly.

Prompt :

Why does Quick Sort perform poorly on already sorted data in some implementations, and how can this be improved?

Explanation :

Quick Sort can perform poorly on already sorted data in some implementations due to the choice of the **pivot element**.

The poor performance of Quick Sort on already sorted data is primarily due to unbalanced partitions caused by poor pivot selection. Using randomized pivot selection or the median-of-three method significantly improves its performance, ensuring that the time complexity remains closer to the average case of (O(n \log n)).

Code :

```python
def quick_sort(arr):
    """
    Sorts an array using the Quick Sort algorithm.
    Parameters:
    arr (list): The list of elements to be sorted.
    Returns:
    list: A new sorted list containing the same elements as arr.
    Recursion:
    The function recursively partitions the array around a pivot element, sorting the
    left and right subarrays until the base case is reached (when the array has 0 or 1 elements).
    """
    if len(arr) <= 1:  # Base case: arrays with 0 or 1 element are already sorted
        return arr
    pivot = arr[len(arr) // 2]  # Choosing the middle element as the pivot
    left = [x for x in arr if x < pivot]  # Elements less than the pivot
    middle = [x for x in arr if x == pivot]  # Elements equal to the pivot
    right = [x for x in arr if x > pivot]  # Elements greater than the pivot
    # Recursive calls to sort left and right subarrays, then concatenate results
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """
    Sorts an array using the Merge Sort algorithm.
    Parameters:
    arr (list): The list of elements to be sorted.
    Returns:
    list: A new sorted list containing the same elements as arr.
    Recursion:
    The function recursively divides the array into halves until it reaches arrays of size 1 or 0
    (base case). Then it merges the sorted halves back together in sorted order.
    """
    if len(arr) <= 1:  # Base case: arrays with 0 or 1 element are already sorted
        return arr
    mid = len(arr) // 2  # Finding the midpoint to divide the array
    left_half = merge_sort(arr[:mid])  # Recursively sort the left half
    right_half = merge_sort(arr[mid:])  # Recursively sort the right half
    # Merging the sorted halves together
    merged = []
    i = j = 0
    while i < len(left_half) and j < len(right_half):
        if left_half[i] < right_half[j]:
            merged.append(left_half[i])
            i += 1
        else:
            merged.append(right_half[j])
            j += 1
    # If there are remaining elements in left_half or right_half, add them to merged
    merged.extend(left_half[i:])
    merged.extend(right_half[j:])
    return merged
# Sample test cases
if __name__ == "__main__":
    # Test case 1: Random list
    random_list = [3, 6, 8, 10, 1, 2, 1]
    print("Quick Sort on random list:", quick_sort(random_list))
    print("Merge Sort on random list:", merge_sort(random_list))
    # Test case 2: Already sorted list
    sorted_list = [1, 2, 3, 4, 5]
    print("Quick Sort on sorted list:", quick_sort(sorted_list))
    print("Merge Sort on sorted list:", merge_sort(sorted_list))
    # Test case 3: Reverse sorted list
    reverse_sorted_list = [5, 4, 3, 2, 1]
    print("Quick Sort on reverse sorted list:", quick_sort(reverse_sorted_list))
    print("Merge Sort on reverse sorted list:", merge_sort(reverse_sorted_list))
```

Out Put :

```
Quick Sort on random list: [1, 1, 2, 3, 6, 8, 10]
Merge Sort on random list: [1, 1, 2, 3, 6, 8, 10]
Quick Sort on sorted list: [1, 2, 3, 4, 5]
Merge Sort on sorted list: [1, 2, 3, 4, 5]
Quick Sort on reverse sorted list: [1, 2, 3, 4, 5]
Merge Sort on reverse sorted list: [1, 2, 3, 4, 5]
```

Task 5: Optimizing a Duplicate Detection Algorithm

Prompt :

I am building a data validation module that must detect duplicate user IDs in a large dataset before importing it into the system.
Task Requirements:
1. First, implement a naive duplicate detection algorithm in Python:
   - Use nested loops (brute-force approach).
   - Return True if duplicates are found, otherwise False.
   - Add inline comments explaining comparisons.
   - Add a proper docstring explaining parameters and return value.
2. Analyze and clearly explain:
   - Time complexity of the brute-force approach.
   - Why it becomes inefficient for large datasets.
3. Suggest a more optimized approach using:
   - Set OR dictionary.
4. Rewrite the algorithm using the optimized method:
   - Add proper docstring.
   - Add clear inline comments explaining why it is faster.
   - Keep it beginner-friendly.
5. Compare both approaches conceptually:
   - Execution behavior for small input sizes.
   - Execution behavior for very large input sizes.
   - Time complexity comparison.
   - Space complexity comparison.
6. Provide sample input and expected output.
Do NOT use built-in functions like list.count().
Keep the explanation simple and academic.

Code :

```python
# Naive duplicate detection algorithm using nested loops
def has_duplicates_naive(user_ids):
    """
    Check for duplicate user IDs using a brute-force approach.

    Parameters:
    user_ids (list): A list of user IDs to check for duplicates.

    Returns:
    bool: True if duplicates are found, False otherwise.
    """
    # Loop through each user ID
    for i in range(len(user_ids)):
        # Compare the current user ID with every other user ID
        for j in range(i + 1, len(user_ids)):
            # If a duplicate is found, return True
            if user_ids[i] == user_ids[j]:
                return True
    # If no duplicates are found after checking all pairs, return False
    return False
# Time complexity analysis:
# The time complexity of the brute-force approach is O(n^2) because we have two
# nested loops, each iterating through the list of user IDs.
# This becomes inefficient for large datasets because as the number of user IDs
# increases, the number of comparisons grows quadratically, leading to significantly longer execution times.
# Optimized duplicate detection algorithm using a set
def has_duplicates_optimized(user_ids):
    """
    Check for duplicate user IDs using an optimized approach with a set.

    Parameters:
    user_ids (list): A list of user IDs to check for duplicates.

    Returns:
    bool: True if duplicates are found, False otherwise.
    """
    # Create an empty set to store unique user IDs
    seen = set()

    # Loop through each user ID
    for user_id in user_ids:
        # If the user ID is already in the set, we have a duplicate
        if user_id in seen:
            return True
        # Otherwise, add the user ID to the set
        seen.add(user_id)

    # If no duplicates are found after checking all user IDs, return False
    return False
# Time complexity analysis:
# The time complexity of the optimized approach is O(n) because we loop through the list of user IDs once,
# and set operations (insertion and lookup) are on average O(1).
# This is much faster than the brute-force approach, especially for large datasets, because we avoid the need
# for nested loops and reduce the number of comparisons significantly.
# Conceptual comparison of both approaches:
# For small input sizes, both approaches may perform similarly, and the difference in execution time may not be noticeable.
# However, as the input size grows, the brute-force approach will take significantly more time due to its
# O(n^2) complexity, while the optimized approach will scale much better with O(n) complexity.
# Space complexity comparison:
# The brute-force approach has a space complexity of O(1) since it uses a constant amount of extra space for the loop variables.
# The optimized approach has a space complexity of O(n) in the worst case, if all user IDs are unique and need to be stored in the set.
# Sample input and expected output
sample_user_ids = ['user1', 'user2', 'user3', 'user1']
print(has_duplicates_naive(sample_user_ids))  # Expected output: True
print(has_duplicates_optimized(sample_user_ids))  # Expected output: True
```

Out Put :

```
True
True
```

Prompt :

Explain mathematically why using a set reduces the time complexity from $O(n^2)$ to $O(n)$.

Explanation :

The use of a set in the has_duplicates_optimized function reduces the time complexity from (O(n^2)) to (O(n)) because of the way sets handle membership checks and insertions. Here's the mathematical reasoning:

Naive Approach: (O(n^2))

1. Nested Loops:

- In the naive implementation, there are two nested loops:
  - The outer loop iterates over all elements ((n) iterations).
  - The inner loop compares the current element with all subsequent elements.
- For each element, the number of comparisons is proportional to the remaining elements in the list: [n-1, n-2, n-3, \dots, 1]
- The total number of comparisons is the sum of the first (n-1) integers: [\text{Total Comparisons} = \frac{(n-1)n}{2} = O(n^2)]
- This quadratic growth makes the algorithm inefficient for large datasets.

2. Optimized Approach: (O(n))

1. Using a Set:

- A set is a data structure that provides **average (O(1))** time complexity for both:
  - **Membership checks** (to see if an element exists in the set).
  - **Insertions** (to add an element to the set).
- This is because sets are typically implemented as hash tables, where operations are performed in constant time on average.

2. Algorithm Steps:

- For each element in the list:
  - Check if the element exists in the set (membership check: (O(1))).
  - If not, add the element to the set (insertion: (O(1))).
- Since there is only one loop iterating through the (n) elements, and each operation inside the loop is (O(1)), the total time complexity is: [O(n) \times O(1) = O(n)]

Why the Time Complexity is Reduced:

- In the naive approach, every pair of elements is compared, leading to (O(n^2)) comparisons.
- In the optimized approach, the use of a set ensures that each element is processed only once, and the operations (membership check and insertion) are constant time on average, resulting in (O(n)) complexity.

Conclusion:

The key improvement comes from replacing the (O(n)) membership check in the inner loop of the naive approach with the (O(1)) membership check provided by the set. This reduces the total number of operations from quadratic ((O(n^2))) to linear ((O(n))), making the optimized approach significantly faster for large datasets.